Лабораторная работа 2. «DEAP - пакет для создания генетических алгоритмов»

DEAP (сокращение от Distributed Evolutionary Algorithms in Python – распределенные эволюционные алгоритмы на Python) разработанного специально для создания реализаций ГА на Python. Подробную документацию можно посмотреть по ссылке:

https://deap.readthedocs.io/en/master/

Первым делом нам нужно его установить. Делается это просто, с помощью команды:

pip install deap

И если все прошло в штатном режиме, то в программе будут работать три следующих модуля:

```
from deap import base
from deap import creator
from deap import tools
```

Начнем с модуля **creator**. Он позволяет создавать новые объекты (классы) в программе. Для этого используется функция:

create(<название класса>, <базовый класс>, [атрибуты нового класса])

Если эту функцию записать (1):

creator.create("Point", object, MAX_COORD = 10, MIN_COORD = 0, coords=list)

Название класса - "Point", наследоваться он будет от стандартного класса **- object,** затем создаем три свойства **MAX_COORD = 10, MIN_COORD =0, coords=list** (coords – это будет просто список языка Python list). После выполнения этой функции в ветке **creator** будет создан класс вида (2):

```
class Point(object):
    MAX_COORD = 10
    MIN_COORD = 0

    def __init__(self):
        self.coords = list()
```

MAX_COORD = 10 и MIN_COORD = 0 в (см. (1)) являются атрибутами самого класса (2), а свойство **coords=list** (см. (1)) будет являться свойством экземпляра этого класса (**self.coords = list()**) (см. (2)).

Все неизменяемые свойства **MAX_COORD = 10, MIN_COORD = 0**, (см. (1)), которые мы прописываем они становятся атрибутами класса

MAX_COORD = 10
MIN_COORD = 0 (см. (2)),

а изменяемые свойства **list** (см. (1)) становятся свойствами его экземпляров **self.coords = list()** (см. (2)).

После того как класс создан его можно использовать, например, так:

pt = creator.Point()

т.е. обратится к ветки **creator** , затем к классу **Point** и создать, например, его экземпляр.

Рассмотрим, как использовать эту функцию в нашей программе. Ранее в программе (Лабораторная работа 2) были определены два класса

class FitnessMAX() и **class Individual(list)**

```
16
17  class FitnessMax():
18      def __init__(self):
19          self.values = [0]
20
21
22  class Individual(list):
23      def __init__(self, *args):
24          super().__init__(*args)
25          self.fitness = FitnessMax()
26
```

Теперь, определим их через функцию **create.** Первое, мы импортируем три модуля в нашу программу

```
from deap import base
from deap import creator
from deap import tools
```

Т.е.

```
ex1.py
1  from deap import base
2  from deap import creator
3  from deap import tools
4
5  import random
6  import matplotlib.pyplot as plt
7
```

Далее, вместо **class FitnessMAX()** прописываем функции **create**.

```
20
21 # class FitnessMax():
22 #     def __init__(self):
23 #         self.values = [0]
24
25
26 class Individual(list):
27     def __init__(self, *args):
28         super().__init__(*args)
29         self.fitness = FitnessMax()
30
31 creator.create("FitnessMax", base.Fitness, weights=(1.0,))
32
```

Первый класс запишем в виде:

creator.create("FitnessMax", base.Fitness, weights=(1.0,))

Здесь мы создаем класс с именем **FitnessMax**, наследуя от базового класса **base.Fitness**, который отвечает за операции над значениями приспособленности конкретного индивида. В частности, параметр **weights** со значением **1.0** означает, что мы будем использовать одно число для оценки степени приспособленности

особи, и это число нам нужно максимизировать (т.е. будем искать максимум функции приспособленности).

Когда мы прописали наш класс **FitnessMax**:

creator.create("FitnessMax", base.Fitness, weights=(1.0,))

в действительности значение приспособленности особей будет определятся одним числом, вес этого числа мы взяли **1.0**. В результате наше конкретное значение приспособленности особей будет вычисляться по формуле:

$$values * weights = (values[0] \cdot weights[0], values[1] \cdot weights[1], ...)$$

values – это значение приспособленностей, которое мы можем рассчитать на основе неких функций (собственных функций, которые задаем в программе).

weights – это веса с которыми мы взвешиваем отдельные значения приспособленности особей.

В итоги получаем кортеж значений

$$(values[0] \cdot weights[0], values[1] \cdot weights[1], ...)$$

т.е. каждое свойство **values[0]**, которое есть в классе **FitnessMax** оно умножается на вес **weights[0]** и формируется некоторое окончательное значение, т.е. если вес равен **1.0,** то его здесь, как бы нет **weights[0].** Мы берем то значение приспособленности функции которое есть.

Если вес будет **-1.0**

creator.create("FitnessMax", base.Fitness, weights=(-1.0,))

то, здесь появится значение -1,

$$values * \overset{-1}{weights} = (values[0] \cdot weights[0], values[1] \cdot weights[1], ...)$$

и в действительности **values[0]** будет отрицательное значение. Это может быть использовано в задачах, где необходимо минимизировать некоторую функцию, но по умолчанию генетический алгоритм максимизирует приспособленность. Чтобы была возможность применить генетический алгоритм функцию **values[0]** будем

умножать на -1. Таким образом генетический алгоритм будет максимизировать некоторое значение, но при этом сама функция будет стремится к минимуму, благодаря весу -1. Через веса (**weights=(1.0,) weights=(-1.0,))** можно управлять оптимизацией.

В задачи **OneMax** значение принадлежности индивида определяется как сумма значений его генов:

$$values[0] = \sum_{i=0}^{N-1} individ[i]$$

Имеем одно число **values[0]** и один вес **weights[0]**, то есть, итоговое значение функции принадлежности, будет равно:

$$f(individ) = \left(\sum_{i=0}^{N-1} individ[i] \right) \cdot 1,0 = \sum_{i=0}^{N-1} individ[i]$$

Теперь определим **class Individual(list)**

```
25
26  class Individual(list):
27      def __init__(self, *args):
28          super().__init__(*args)
29          self.fitness = FitnessMax()
30
31  creator.create("FitnessMax", base.Fitness, weights=(1.0,))
32
```

прописываем **creator.create("Individual", list, fitness=creator.FitnessMax)**

```
25
26  class Individual(list):
27      def __init__(self, *args):
28          super().__init__(*args)
29          self.fitness = FitnessMax()
30
31  creator.create("FitnessMax", base.Fitness, weights=(1.0,))
32  creator.create("Individual", list, fitness=creator.FitnessMax)
33
```

При этом **class Individual(list)** ставим в комментарий

```
ex1.py
21  # class FitnessMax():
22  #     def __init__(self):
23  #         self.values = [0]
24
25
26  # class Individual(list):
27  #     def __init__(self, *args):
28  #         super().__init__(*args)
29  #         self.fitness = FitnessMax()
30
31  creator.create("FitnessMax", base.Fitness, weights=(1.0,))
32  creator.create("Individual", list, fitness=creator.FitnessMax)
33
```

Класс **Individual**, наследуется от класса **list** и имеет одно локальное свойство **fitness**, который будет являться экземпляром ранее созданного класса **FitnessMax.** Для возможность к нему обратиться следует указать ветку **creator** в котором этот класс создан. Причем свойство **fitness** будет являться локальным свойством экземпляра класса I**ndividual**, потому, что класс **FitnessMax** воспринимается как изменяемый объект. Следовательно, **fitness** будет являться локальным свойством экземпляров класса **Individual.**

Далее, в программе (Лабораторная работа 1) было определено несколько вспомогательных функций.

```
33
34  def oneMaxFitness(individual):
35      return sum(individual), # кортеж
36
37  def individualCreator():
38      return Individual([random.randint(0, 1) for i in range(ONE_MAX_LENGTH)])
39
40  def populationCreator(n = 0):
41      return list([individualCreator() for i in range(n)])
42
```

Определим, как можно с помощью пакета **DEAP** упростить этот фрагмент программы. Для этого воспользуемся классом

base.Toolbox

и модулем:

tools

Пусть есть некоторая функция (3)для вычисления суммы двух величин:

```
def sum_two(a, b):
return a+b
```

Тогда, с помощью экземпляра класса **base.Toolbox()**:

```
toolbox = base.Toolbox()
```

можно зарегистрировать псевдоним этой функции (3), следующим образом:

```
toolbox.register("sum_alias", sum_two)
```

(**register** – метод, **"sum_alias"** – новое имя функции (3), **sum_two –** ссылка на функцию (3)).

Таким образом, мы можем обратиться к функции (3) через экземпляр класса **toolbox** следующим образом:

```
res = toolbox. sum_alias(5, 2)
```

значение **res =7**.

Здесь мы не получаем особо никакого дополнительного эффекта. Но можно зарегистрировать псевдоним и так, указывая, что второй параметр **b** по умолчанию принимает значение **2**:

```
toolbox.register("sum_2", sum_two, b = 2)
```

(**register** – метод, "sum_2" – новый псевдоним, sum_two – функция связанная с псевдонимом, в данном случае это функция (3) и, кроме того **b=2**). Тогда

вызвать функцию (3) можно следующим образом (при этом указываем только одно значение **a=5**):

res = toolbox.sum_2(5)

В итоги происходит суммирование 5+2=7, т.е. **res =7.** Именно эти дополнительные параметры, которые можно указывать для алиаса функции и делают инструмент **toolbox** полезным при разработке генетических алгоритмов.

Вернемся в нашу программу

```
31  creator.create("FitnessMax", base.Fitness, weights=(1.0,))
32  creator.create("Individual", list, fitness=creator.FitnessMax)
33
34  def oneMaxFitness(individual):
35      return sum(individual), # кортеж
36
37  def individualCreator():
38      return Individual([random.randint(0, 1) for i in range(ONE_MAX_LENGTH)])
39
40  def populationCreator(n = 0):
41      return list([individualCreator() for i in range(n)])
42
43
```

Функцию o*neMaxFitness* мы оставим без изменения (эта функция вычисляет приспособленность конкретного individual:

def oneMaxFitness(individual):

return sum(individual), # кортеж

А следующие функции мы заменим.

```
33
34  def oneMaxFitness(individual):
35      return sum(individual), # кортеж
36
37  # def individualCreator():
38  #     return Individual([random.randint(0, 1) for i in range(ONE_MAX_LENGTH)])
39  #
40  # def populationCreator(n = 0):
41  #     return list([individualCreator() for i in range(n)])
42
```

Пропишем следующие строчки, т.е. экземпляр класса base.Toolbox, зарегистрируем с начало функцию **zeroOrOne** (т.е. функция, которая будет являться псевдонимом стандартной функции **random.randint**), **random.randint**, генерирует случайные значения 0 или 1.

```
36
37  # def individualCreator():
38  #     return Individual([random.randint(0, 1) for i in range(ONE_MAX_LENGTH)])
39  #
40  # def populationCreator(n = 0):
41  #     return list([individualCreator() for i in range(n)])
42
43  toolbox = base.Toolbox()
44  toolbox.register("zeroOrOne", random.randint, 0, 1)
45
```

Далее определим функцию:

toolbox.register("individualCreator", tools.initRepeat, creator.Individual, toolbox.zeroOrOne, ONE_MAX_LENGTH)

```
42
43 toolbox = base.Toolbox()
44 toolbox.register("zeroOrOne", random.randint, 0, 1)
45 toolbox.register("individualCreator", tools.initRepeat, creator.Individual, toolbox.zeroOrOne, ONE_MAX_LENGTH)
```

Функция "individualCreator" для создания списка состоящего из случайных значений **0** и **1**. Для этого вызывается стандартная функция **initRepeat**, модуля **tools**, которая и позволяет создавать списки.

Как она работает

tools.initRepeat(<контейнер для хранения генов>, <функция генерации значения гена>, <число генов в хромосоме>)

Функция **initRepeat** воспринимает три параметра: **<контейнер для хранения генов>, <функция генерации значения гена>, <число генов в хромосоме>**

Именно эти три параметра и указаны для данной функции: **creator.Individual, toolbox.zeroOrOne, ONE_MAX_LENGTH**

Класс **Individual** наследуется от базового класса **list,** т.е. будет формироваться обычный стандартный список языка Python. Каждое значение этого списка будет формироваться на основе функции **zeroOrOne** (т.е . это будет случайное число 0 или 1, а количество таких чисел в списке будет равняться **ONE_MAX_LENGTH** т.е. длина хромосомы).

```
30
31 creator.create("FitnessMax", base.Fitness, weights=(1.0,))
32 creator.create("Individual", list, fitness=creator.FitnessMax)
33
34 def oneMaxFitness(individual):
35     return sum(individual), # кортеж
36
37 # def individualCreator():
38 #     return Individual([random.randint(0, 1) for i in range(ONE_MAX_LENGTH)])
39 #
40 # def populationCreator(n = 0):
41 #     return list([individualCreator() for i in range(n)])
42
43 toolbox = base.Toolbox()
44 toolbox.register("zeroOrOne", random.randint, 0, 1)
45 toolbox.register("individualCreator", tools.initRepeat, creator.Individual, toolbox.zeroOrOne, ONE_MAX_LENGTH)
```

Пропишем следующую функцию для генерации всей популяции **toolbox.register("populationCreator", tools.initRepeat, list, toolbox.individualCreator)**

```
42
43 toolbox = base.Toolbox()
44 toolbox.register("zeroOrOne", random.randint, 0, 1)
45 toolbox.register("individualCreator", tools.initRepeat, creator.Individual, toolbox.zeroOrOne, ONE_MAX_LENGTH)
46 toolbox.register("populationCreator", tools.initRepeat, list, toolbox.individualCreator)
```

Здесь также используется **initRepeat** , указывается контейнер **list** , который будет содержать список (обычный список языка Python), далее функция **individualCreator**, которая генерирует каждый элемент этого списка. Параметр **ONE_MAX_LENGTH** здесь указывать не будем. Его укажем далее (**n=POPULATION_SIZE**), когда вызовем эту функцию.

population = toolbox.populationCreator(n=POPULATION_SIZE)

Функцию **populationCreator** вызываем через экземпляр класса **toolbox**, далее указываем размер популяции, который создаем **n=POPULATION_SIZE.**

При этом функцию

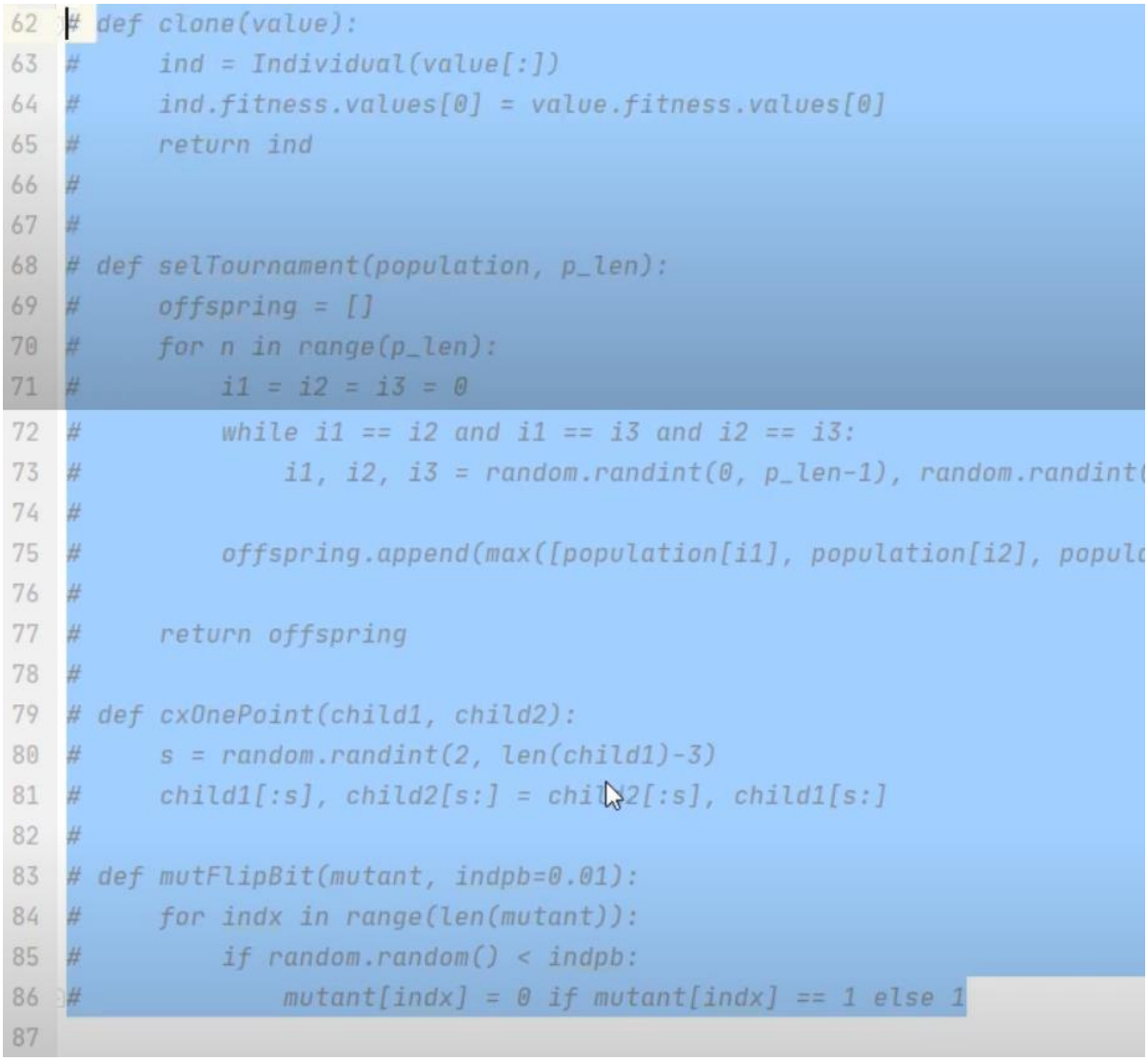
population = populationCreator(n=POPULATION_SIZE)

поставим в комментарий.

```
42
43 toolbox = base.Toolbox()
44 toolbox.register("zeroOrOne", random.randint, 0, 1)
45 toolbox.register("individualCreator", tools.initRepeat, creator.Individual, toolbox.zeroOrOne, ONE_MAX_LENGTH)
46 toolbox.register("populationCreator", tools.initRepeat, list, toolbox.individualCreator)
47
48 population = toolbox.populationCreator(n=POPULATION_SIZE)
49
50 #population = populationCreator(n=POPULATION_SIZE)
```

Далее, вместо прежних наших функций **clone()** (функция клонирования индивидуума), **selTournament()** (турнирный отбор), **cxOnePoint()** (одноточечное скрещивание) и **mutFlipBit()** (мутация) воспользуемся аналогичными встроенными функциями из модуля **tools** из пакета **DEAP,** а эти функции поставим в комментарий.

```
61
62 def clone(value):
63     ind = Individual(value[:])
64     ind.fitness.values[0] = value.fitness.values[0]
65     return ind
66
67
68 def selTournament(population, p_len):
69     offspring = []
70     for n in range(p_len):
71         i1 = i2 = i3 = 0
72         while i1 == i2 and i1 == i3 and i2 == i3:
73             i1, i2, i3 = random.randint(0, p_len-1), random.randint(0, p_len-1), random.randint(0, p_len-1)
74
75         offspring.append(max([population[i1], population[i2], population[i3]], key=lambda ind: ind.fitness.va
76
77     return offspring
78
79 def cxOnePoint(child1, child2):
80     s = random.randint(2, len(child1)-3)
81     child1[:s], child2[s:] = child2[:s], child1[s:]
82
83 def mutFlipBit(mutant, indpb=0.01):
84     for indx in range(len(mutant)):
85         if random.random() < indpb:
86             mutant[indx] = 0 if mutant[indx] == 1 else 1
87
```



```
62  # def clone(value):
63  #     ind = Individual(value[:])
64  #     ind.fitness.values[0] = value.fitness.values[0]
65  #     return ind
66  #
67  #
68  # def selTournament(population, p_len):
69  #     offspring = []
70  #     for n in range(p_len):
71  #         i1 = i2 = i3 = 0
72  #         while i1 == i2 and i1 == i3 and i2 == i3:
73  #             i1, i2, i3 = random.randint(0, p_len-1), random.randint(
74  #
75  #         offspring.append(max([population[i1], population[i2], popula
76  #
77  #     return offspring
78  #
79  # def cxOnePoint(child1, child2):
80  #     s = random.randint(2, len(child1)-3)
81  #     child1[:s], child2[:s] = child2[:s], child1[s:]
82  #
83  # def mutFlipBit(mutant, indpb=0.01):
84  #     for indx in range(len(mutant)):
85  #         if random.random() < indpb:
86  #             mutant[indx] = 0 if mutant[indx] == 1 else 1
87
```

Напишем следующее – регистрируем функцию с именем **"select"** , которая будет ссылаться на стандартную функцию **selTournament** (т.е. турнирный отбор). Далее указываем количество особей, участвующих в турнирном отборе **tournsize=3**. Пусть их будет 3. (Также, как это было раньше – 3 особи).

Аналогично для скрещивания. Функция **cxOnePoint** выполнять одноточечное скрещивание для наших родителей. Алиас будет **"mate".**

Для мутации функция **mutFlipBit** (из пакета **DEAP**) будет инвертировать отдельные биты генов. Параметр **indpb** определяет вероятность мутации отдельного гена в хромосоме выбранной особи.

Обратите внимание, название алиасов (псевдонимов) этих функций лучше записывать именно в таком виде: **select**, **mate** и **mutate**, так как эти имена, затем, используются функциями пакета **DEAP** при работе ГА.

```
86  #           mutant[indx] = 0 if mutant[indx] == 1 else 1
87
88  toolbox.register("select", tools.selTournament, tournsize=3)
89  toolbox.register("mate", tools.cxOnePoint)
90  toolbox.register("mutate", tools.mutFlipBit, indpb=1.0/ONE_MAX_LENGTH)
91
```

Еще необходимо зарегистрировать функцию с именем **evaluate**, которая должна возвращать кортеж значений приспособленности каждого отдельного индивида (т.е. функция **evaluate** будет ссылаться на функцию **oneMaxFitness**, которую выше объявили, для расчёта приспособленности каждого отдельного индивидуума

```
33
34  def oneMaxFitness(individual):
35      return sum(individual), # кортеж
36
```

. При этом возвращается кортеж, а не отдельные значения.)

```
87
88  toolbox.register("evaluate", oneMaxFitness)
89  toolbox.register("select", tools.selTournament, tournsize=3)
90  toolbox.register("mate", tools.cxOnePoint)
91  toolbox.register("mutate", tools.mutFlipBit, indpb=1.0/ONE_MAX_LENGTH)
92
```

Таким образом, использование стандартных функций пакета DEAP значительно сокращает программу.

Ранее записанные строчки переводим в комментарий:

```
87
88  toolbox.register("evaluate", oneMaxFitness)
89  toolbox.register("select", tools.selTournament, tournsize=3)
90  toolbox.register("mate", tools.cxOnePoint)
91  toolbox.register("mutate", tools.mutFlipBit, indpb=1.0/ONE_MAX_LENGTH)
92
93
94  # fitnessValues = [individual.fitness.values[0] for individual in population]
95  #
96  # while max(fitnessValues) < ONE_MAX_LENGTH and generationCounter < MAX_GENERATIONS:
97  #     generationCounter += 1
98  #     offspring = selTournament(population, len(population))
99  #     offspring = list(map(clone, offspring))
```

```
97      #     offspring = selTournament(population, len(population))
98      #     offspring = list(map(clone, offspring))
99      #
100     #     for child1, child2 in zip(offspring[::2], offspring[1::2]):
101     #         if random.random() < P_CROSSOVER:
102     #             cxOnePoint(child1, child2)
103     #
104     #     for mutant in offspring:
105     #         if random.random() < P_MUTATION:
106     #             mutFlipBit(mutant, indpb=1.0/ONE_MAX_LENGTH)
107     #
108     #     freshFitnessValues = list(map(oneMaxFitness, offspring))
109     #     for individual, fitnessValue in zip(offspring, freshFitnessValues):
110     #         individual.fitness.values = fitnessValue
111     #
112     #     population[:] = offspring
113     #
114     #     fitnessValues = [ind.fitness.values[0] for ind in population]
115     #
116     #     maxFitness = max(fitnessValues)
117     #     meanFitness = sum(fitnessValues) / len(population)
```

```
#     meanFitness = sum(fitnessValues) / len(population)
#     maxFitnessValues.append(maxFitness)
#     meanFitnessValues.append(meanFitness)
#     print(f"Поколение {generationCounter}: Макс приспособ. = {maxFitness}, Средняя приспособ
#
#     best_index = fitnessValues.index(max(fitnessValues))
#     print("Лучший индивидуум = ", *population[best_index], "\n")

plt.plot(maxFitnessValues, color='red')
plt.plot(meanFitnessValues, color='green')
plt.xlabel('Поколение')
plt.ylabel('Макс/средняя приспособленность')
plt.title('Зависимость максимальной и средней приспособленности от поколения')
plt.show()
```

Далее, для запуска ГА воспользуемся готовой функцией **eaSimple()** модуля **algorithms**, который вначале также импортируем:

```
from deap import base, algorithms
population, logbook = algorithms.eaSimple(population, toolbox,
                                          cxpb=P_CROSSOVER,
                                          mutpb=P_MUTATION,
                                          ngen=MAX_GENERATIONS,
                                          verbose=True)
```

```
toolbox.register("evaluate", oneMaxFitness)
toolbox.register("select", tools.selTournament, tournsize=3)
toolbox.register("mate", tools.cxOnePoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=1.0/ONE_MAX_LENGTH)

population, logbook = algorithms.eaSimple(population, toolbox,
                                          cxpb=P_CROSSOVER,
                                          mutpb=P_MUTATION,
                                          ngen=MAX_GENERATIONS,
                                          verbose=True)
```

В этой функции в качестве параметров передаем начальную популяцию (**population**), набор объявленных функций модуля (**toolbox**), затем, указываем вероятности для операции скрещивания (**P_CROSSOVER**) и мутации (**P_MUTATION**), максимальное количество итераций работы генетического алгоритма (**MAX_GENERATIONS**). Параметр **verbose=True** означает выводить служебную информацию на каждой итерации работы алгоритма. На выходе эта функция дает конечную популяцию и служебную информацию в виде объекта **logbook.**

Причем функция **eaSimple** возвращает конечную популяцию (**population**), которая получится и служебный объект **logbook** (в нем хранится статистика, которая собирается на каждой итерации работы алгоритма).

Весь ГА готов:

x1.py

```
from deap import base, algorithms
from deap import creator
from deap import tools

import random
import matplotlib.pyplot as plt

# константы задачи
ONE_MAX_LENGTH = 100    # длина подлежащей оптимизации битовой строки

# константы генетического алгоритма
POPULATION_SIZE = 200   # количество индивидуумов в популяции
P_CROSSOVER = 0.9       # вероятность скрещивания
P_MUTATION = 0.1        # вероятность мутации индивидуума
MAX_GENERATIONS = 50    # максимальное количество поколений

RANDOM_SEED = 42
random.seed(RANDOM_SEED)


# class FitnessMax():
```

```
#     def __init__(self):
#         self.values = [0]

#
# class Individual(list):
#     def __init__(self, *args):
#         super().__init__(*args)
#         self.fitness = FitnessMax()

creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)

def oneMaxFitness(individual):
    return sum(individual), # кортеж

# def individualCreator():
#     return Individual([random.randint(0, 1) for i in range(ONE_MAX_LENGTH)])
#
# def populationCreator(n = 0):
#     return list([individualCreator() for i in range(n)])

toolbox = base.Toolbox()
```

ex1.py

```
toolbox = base.Toolbox()
toolbox.register("zeroOrOne", random.randint, 0, 1)
toolbox.register("individualCreator", tools.initRepeat, creator.Individual, toolbox.zeroOrOne, ONE_MAX_LENGTH)
toolbox.register("populationCreator", tools.initRepeat, list, toolbox.individualCreator)

population = toolbox.populationCreator(n=POPULATION_SIZE)

#population = populationCreator(n=POPULATION_SIZE)
generationCounter = 0

fitnessValues = list(map(oneMaxFitness, population))

for individual, fitnessValue in zip(population, fitnessValues):
    individual.fitness.values = fitnessValue

maxFitnessValues = []
meanFitnessValues = []


# def clone(value):
#     ind = Individual(value[:])
```

1.py

```
#     ind.fitness.values[0] = value.fitness.values[0]
#     return ind
#
#
# def selTournament(population, p_len):
#     offspring = []
#     for n in range(p_len):
#         i1 = i2 = i3 = 0
#         while i1 == i2 and i1 == i3 and i2 == i3:
#             i1, i2, i3 = random.randint(0, p_len-1), random.randint(0, p_len-1), random.randint(0, p
#
#         offspring.append(max([population[i1], population[i2], population[i3]], key=lambda ind: ind.f
#
#     return offspring
#
# def cxOnePoint(child1, child2):
#     s = random.randint(2, len(child1)-3)
#     child1[:s], child2[s:] = child2[:s], child1[s:]
#
# def mutFlipBit(mutant, indpb=0.01):
#     for indx in range(len(mutant)):
#         if random.random() < indpb:
```

```
ex1.py
85  #         if random.random() < indpb:
86  #             mutant[indx] = 0 if mutant[indx] == 1 else 1
87
88  toolbox.register("evaluate", oneMaxFitness)
89  toolbox.register("select", tools.selTournament, tournsize=3)
90  toolbox.register("mate", tools.cxOnePoint)
91  toolbox.register("mutate", tools.mutFlipBit, indpb=1.0/ONE_MAX_LENGTH)
92
93  population, logbook = algorithms.eaSimple(population, toolbox,
94                                            cxpb=P_CROSSOVER,
95                                            mutpb=P_MUTATION,
96                                            ngen=MAX_GENERATIONS,
97                                            verbose=True)
98
99  # fitnessValues = [individual.fitness.values[0] for individual in population]
100 #
101 # while max(fitnessValues) < ONE_MAX_LENGTH and generationCounter < MAX_GENERATIONS:
102 #     generationCounter += 1
103 #     offspring = selTournament(population, len(population))
104 #     offspring = list(map(clone, offspring))
105 #
```

```
ex1.py
106 #     for child1, child2 in zip(offspring[::2], offspring[1::2]):
107 #         if random.random() < P_CROSSOVER:
108 #             cxOnePoint(child1, child2)
109 #
110 #     for mutant in offspring:
111 #         if random.random() < P_MUTATION:
112 #             mutFlipBit(mutant, indpb=1.0/ONE_MAX_LENGTH)
113 #
114 #     freshFitnessValues = list(map(oneMaxFitness, offspring))
115 #     for individual, fitnessValue in zip(offspring, freshFitnessValues):
116 #         individual.fitness.values = fitnessValue
117 #
118 #     population[:] = offspring
119 #
120 #     fitnessValues = [ind.fitness.values[0] for ind in population]
121 #
122 #     maxFitness = max(fitnessValues)
123 #     meanFitness = sum(fitnessValues) / len(population)
124 #     maxFitnessValues.append(maxFitness)
125 #     meanFitnessValues.append(meanFitness)
126 #     print(f"Поколение {generationCounter}: Макс приспособ. = {maxFitness}, Средняя приспособ.= {me
```

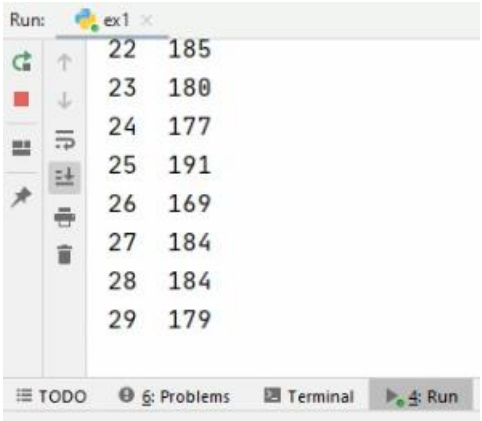
Далее следующие строки

```
130
131
132     plt.plot(maxFitnessValues, color='red')
133     plt.plot(meanFitnessValues, color='green')
134     plt.xlabel('Поколение')
135     plt.ylabel('Макс/средняя приспособленность')
136     plt.title('Зависимость максимальной и средней приспособленности от поколения')
137     plt.show()
138
```

переводим в комментарии, так как нет этих параметров.

```
130
131
132     # plt.plot(maxFitnessValues, color='red')
133     # plt.plot(meanFitnessValues, color='green')
134     # plt.xlabel('Поколение')
135     # plt.ylabel('Макс/средняя приспособленность')
136     # plt.title('Зависимость максимальной и средней приспособленности от поколения')
137     # plt.show()
138
```

Запускаем программу – все работает. Первый столбец – это количество итераций (популяций).



Добавим в качестве статистики максимальное значение особей (**maxFitnessValues**) и среднее значение (**meanFitnessValues**). Для этого используется специальный класс Statistics(), где через **lambda** функцию говорим, какие данные будем брать из отдельного индивидуума для сбора статистики. В данном случае будем брать значение его приспособленности (values).

stats = tools.Statistics(lambda ind: ind.fitness.values)

```
toolbox.register("evaluate", oneMaxFitness)
toolbox.register("select", tools.selTournament, tournsize=3)
toolbox.register("mate", tools.cxOnePoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=1.0/ONE_MAX_LENGTH)

stats = tools.Statistics(lambda ind: ind.fitness.values)

population, logbook = algorithms.eaSimple(population, toolbox,
                                          cxpb=P_CROSSOVER,
                                          mutpb=P_MUTATION,
                                          ngen=MAX_GENERATIONS,
                                          verbose=True)

# fitnessValues = [individual.fitness.values[0] for individual in population]
```

А, затем, в экземпляре этого класса зарегистрируем две функции для выбора максимальной и средней приспособленности особей популяции на каждой итерации работы алгоритма:

stats.register("max", np.max)

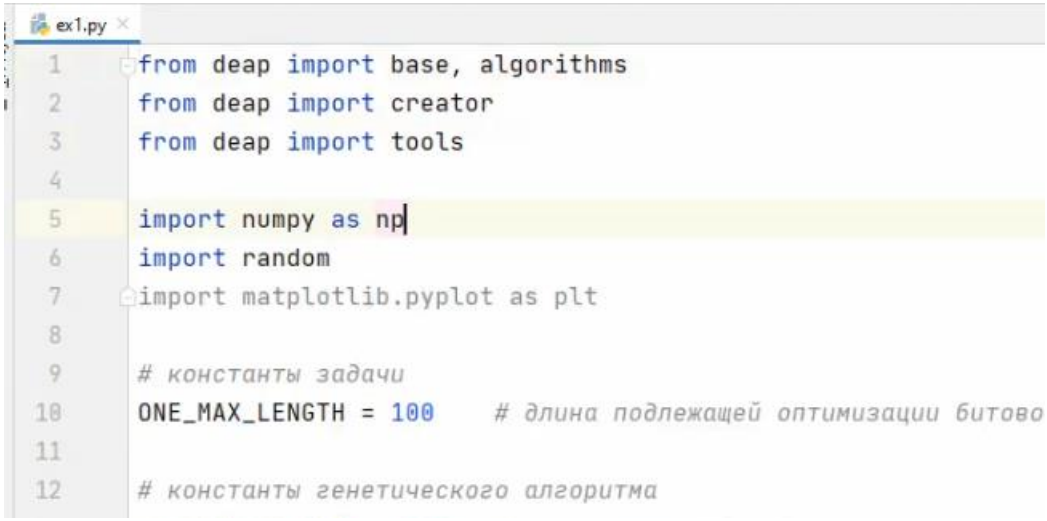
stats.register("avg", np.mean)

```
toolbox.register("evaluate", oneMaxFitness)
toolbox.register("select", tools.selTournament, tournsize=3)
toolbox.register("mate", tools.cxOnePoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=1.0/ONE_MAX_LENGTH)

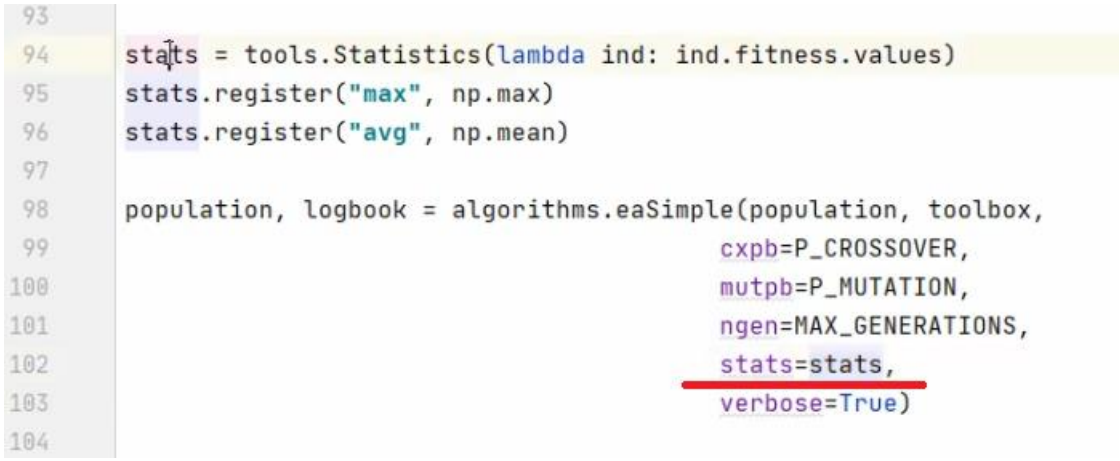
stats = tools.Statistics(lambda ind: ind.fitness.values)
stats.register("max", np.max)
stats.register("avg", np.mean)

population, logbook = algorithms.eaSimple(population, toolbox,
                                          cxpb=P_CROSSOVER,
                                          mutpb=P_MUTATION,
                                          ngen=MAX_GENERATIONS,
                                          verbose=True)
```

np.max, **np.mean** – это функции пакета **numpy**, которые вычисляют максимальное и среднее значение. Импортируем пакет **numpy.**



В функции **eaSimple()** дополнительно укажем параметр **stats=stats,** который будет ссылаться на объект класса **Statistics.**



Чтобы мы могли воспользоваться статистикой добавим запись и выберем из объекта logbook эти два свойства "max" и "avg". Они будут помещены в коллекции **maxFitnessVelues** и **meanFitnessVelues**.

```
135
136 maxFitnessValues, meanFitnessValues = logbook.select("max", "avg")
137
138 # plt.plot(maxFitnessValues, color='red')
139 # plt.plot(meanFitnessValues, color='green')
140 # plt.xlabel('Поколение')
141 # plt.ylabel('Макс/средняя приспособленность')
142 # plt.title('Зависимость максимальной и средней приспособленности от поколения')
143 # plt.show()
144
```

Далее выводим их стандартным образом.

```
135
136 maxFitnessValues, meanFitnessValues = logbook.select("max", "avg")
137
138 plt.plot(maxFitnessValues, color='red')
139 plt.plot(meanFitnessValues, color='green')
140 plt.xlabel('Поколение')
141 plt.ylabel('Макс/средняя приспособленность')
142 plt.title('Зависимость максимальной и средней приспособленности от поколения')
143 plt.show()
144
```

Запустим программу.

```
Run:  ex1
   7   184    77   70.215
   8   180    81   72.475
   9   174    82   74.79
  10   172    85   76.685
  11   177    85   78.46
  12   178    87   80.265
  13   178    87   81.8
  14   184    87   83.15
```

Выводится значения максимальной и средней приспособленности.

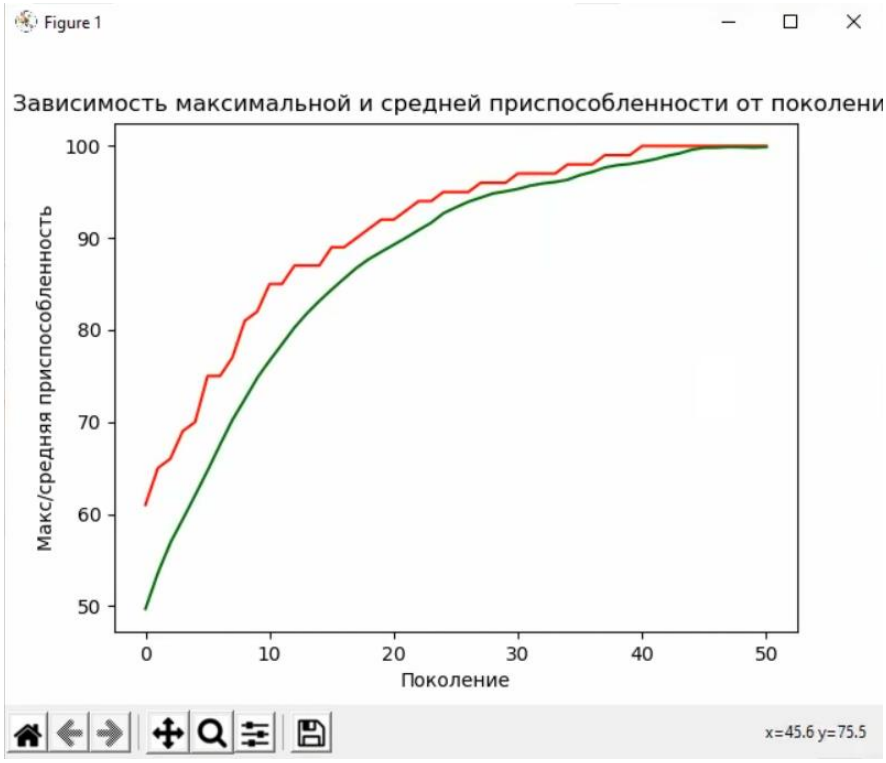
gen	nevals	max	avg
0	0	51	49.695
1	193	65	53.575
2	179	66	56.855
3	187	69	59.42
4	184	70	62.03
5	188	75	64.715
6	176	75	67.515

Максимальная приспособленность равна 100 (аналогично с ЛР 2).

Run: ex1

43	188	100	99.195
44	192	100	99.595
45	193	100	99.82
46	189	100	99.82
47	189	100	99.895
48	190	100	99.875
49	187	100	99.83
50	184	100	99.89

Решение было найдено на 40 итерации.



Программа заметно сократилась. Пакет DEAP значительно упрощает использование ГА.

Задание для самостоятельного выполнения:

У встроенного метода **algorithms.eaSimple** есть еще одна возможность - **зал славы** (hall of fame, сокращенно **hof**). Класс **Hall0fFame**, находящийся в модуле **tools**, позволяет сохранить лучших индивидуумов, встретившихся в процессе эволюции, даже если вследствие отбора, скрещивания и мутации они были в какой-то момент утрачены. Зал славы поддерживается в отсортированном состоянии, так что первым элементом всегда является индивидуум с наилучшим встретившимся значением приспособленности.

Чтобы добавить зал славы, в написанный выше код нужно внести следующие изменения.

1. Определим константу, равную количеству индивидуумов, которых мы хотим хранить в зале славы:

```
HALL_OF_FAME_SIZE = 10
```

2. Прежде чем вызывать алгоритм `eaSimple`, создадим объект `HallOfFame` такого размера:

```
hof = tools.HallOfFame(HALL_OF_FAME_SIZE)
```

3. Объект `HallOfFame` передается алгоритму `eaSimple`, который самостоятельно обновляет его в процессе выполнения генетического алгоритма:

```
population, logbook = algorithms.eaSimple(population, toolbox,
cxpb=P_CROSSOVER, mutpb=P_MUTATION, ngen=MAX_GENERATIONS,
stats=stats, halloffame=hof, verbose=True)
```

4. По завершении алгоритма атрибут `items` объекта `HallOfFame` можно использовать для доступа к списку помещенных в зал славы индивидуумов:

```
print("Индивидуумы в зале славы  = ", *hof.items, sep="\n")
print("Лучший индивидуум = ", hof.items[0])
```

Ниже приведены напечатанные результаты – лучший индивидуум, содержащий только единицы, а после него несколько индивидуумов, содержащих 0 в разных позициях:

```
Индивидуумы в зале славы =
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
...
```

Лучшим является все тот же индивидуум, содержащий одни единицы:

```
Лучший индивидуум = [1, 1, 1, 1, 1, ...
```

Далее мы во всех программах будем использовать эти средства: объекты `Statistics` и `logbook`, встроенный алгоритм и зал славы.

Эксперименты с параметрами алгоритма

Теперь мы можем поэкспериментировать с различными параметрами и определениями и понаблюдать за изменениями поведения и результатов.

В каждом из последующих подразделов мы будем вносить в исходные параметры программы одно или несколько изменений. Призываем вас поэкспериментировать самостоятельно и попробовать подвергнуть программу сразу нескольким модификациям.

Имейте в виду, что последствия изменений могут зависеть от конкретной программы, и то, что мы видим в простой задаче OneMax, может не наблюдаться в других задачах.

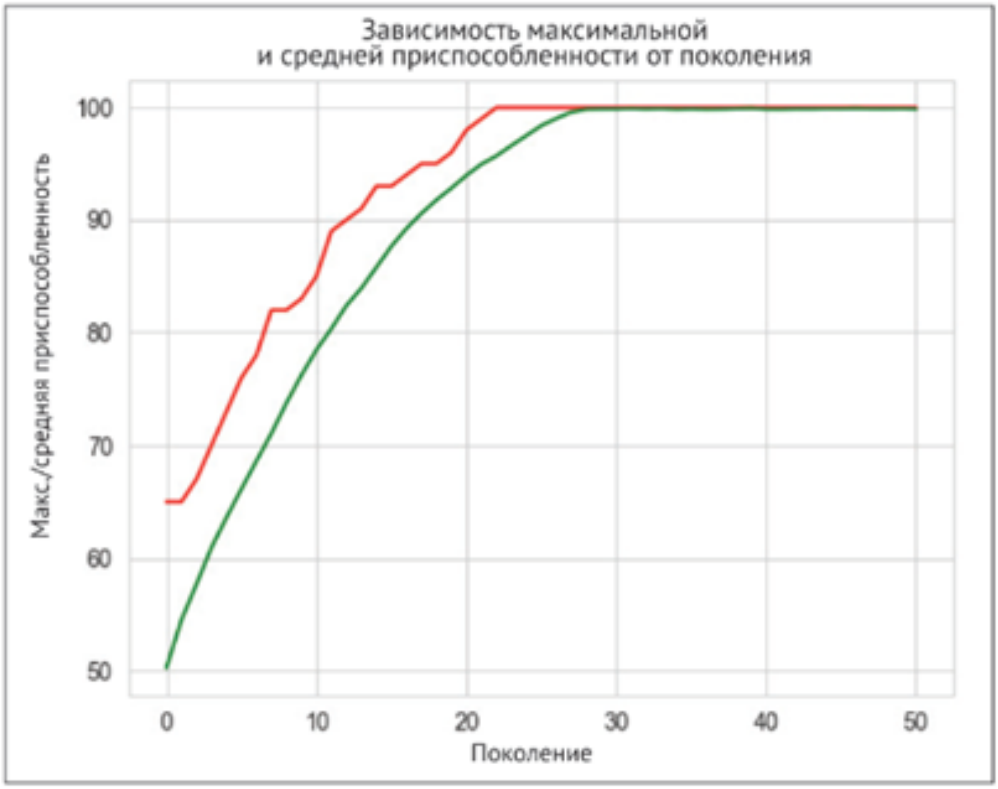
Размер популяции и количество поколений

Для начала изменим размер популяции и количество поколений в генетическом алгоритме.

1. Размер популяции определяется константой `POPULATION_SIZE`. Увеличим ее с 200 до 400:

```
POPULATION_SIZE = 400
```

При этом алгоритм работает быстрее, лучшее решение обнаруживается после 22 поколений, как видно из графика.

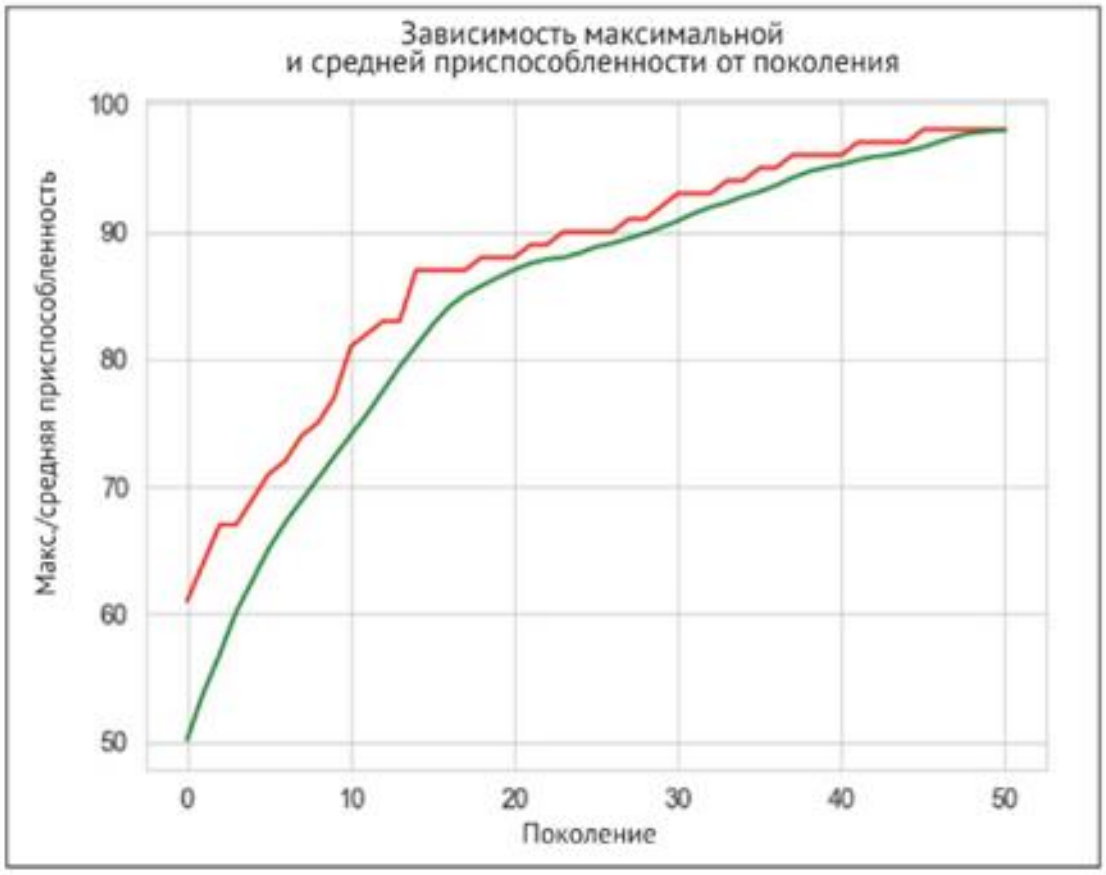


Статистика программы для решения задачи OneMax
после увеличения размера популяции до 400

2. Теперь попробуем уменьшить размер популяции до 100:

```
POPULATION_SIZE = 100
```

Сходимость алгоритма замедляется, на протяжении 50 поколений наилучшее решение так и не найдено.

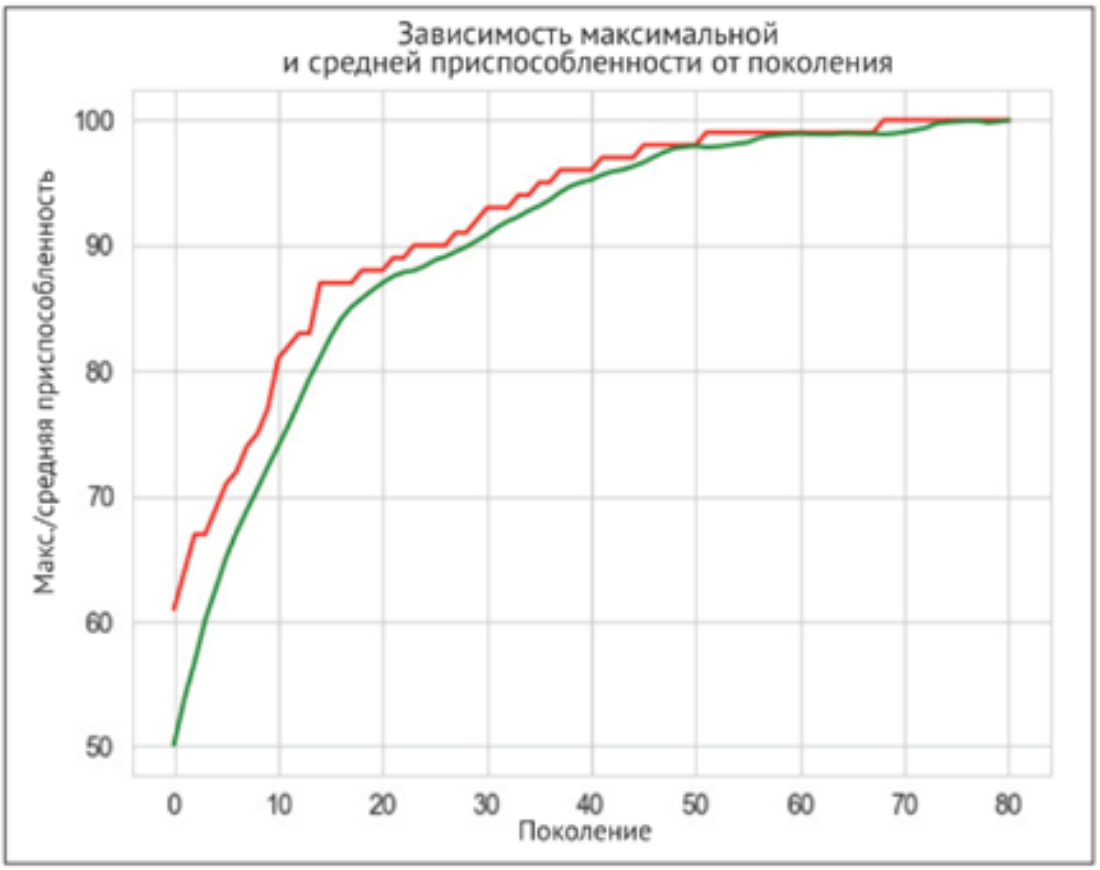


Статистика программы для решения задачи OneMax
после уменьшения размера популяции до 100

3. Для компенсации попробуем увеличить значение `MAX_GENERATIONS` до 80:

```
MAX_GENERATIONS = 80
```

Лучшее решение найдено после 68 поколений.



Статистика программы для решения задачи OneMax
после увеличения количества поколений до 80

Такое поведение типично для генетических алгоритмов – при увеличении популяции для нахождения решения требуется меньше поколений. Однако с ростом размера популяции повышаются требования к вычислительной мощности и памяти, поэтому обычно мы стремимся найти умеренный размер популяции, при котором алгоритм сходится к решению за разумное время.

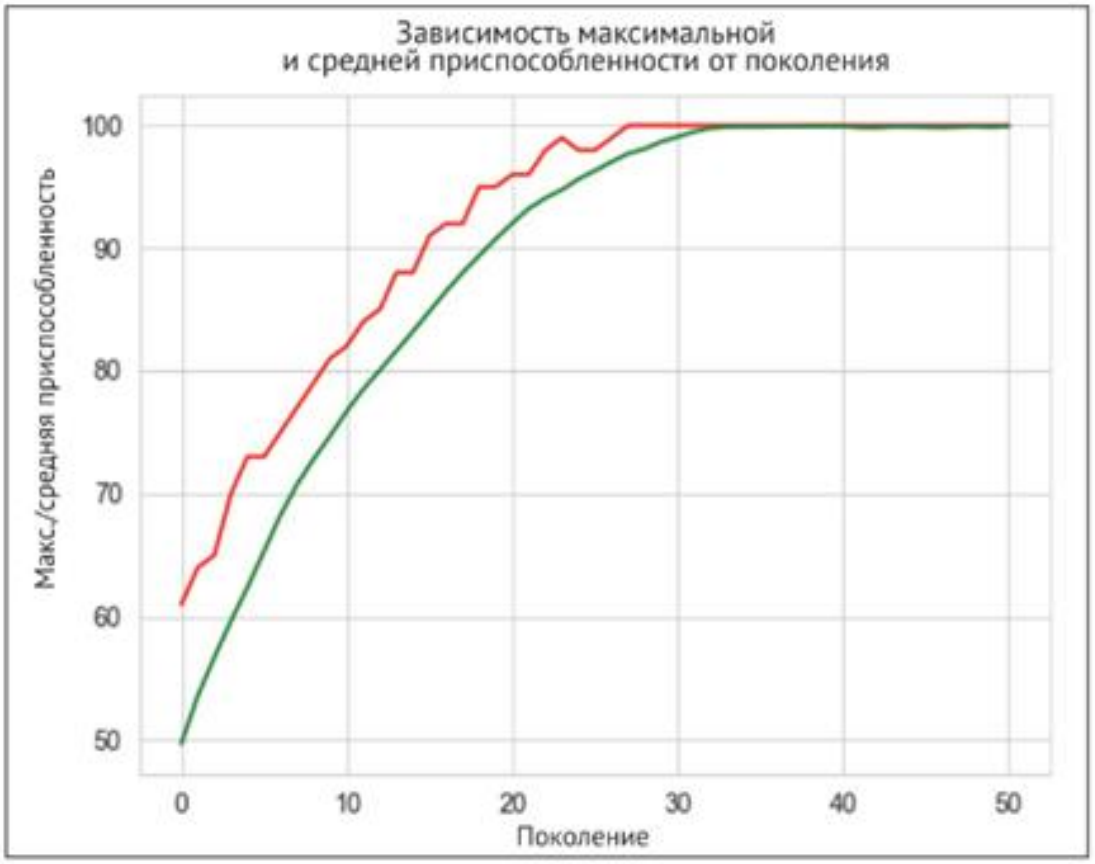
Оператор скрещивания

Откатим изменения и вернемся к первоначальным параметрам (50 поколений, размер популяции 200). Займемся экспериментами с оператором скрещивания, который отвечает за создание потомков.

Для замены одноточечного скрещивания двухточечным нужно лишь определить оператор скрещивания следующим образом:

```
toolbox.register("mate", tools.cxTwoPoint)
```

Теперь алгоритм находит лучшее решение на протяжении всего 27 поколений.



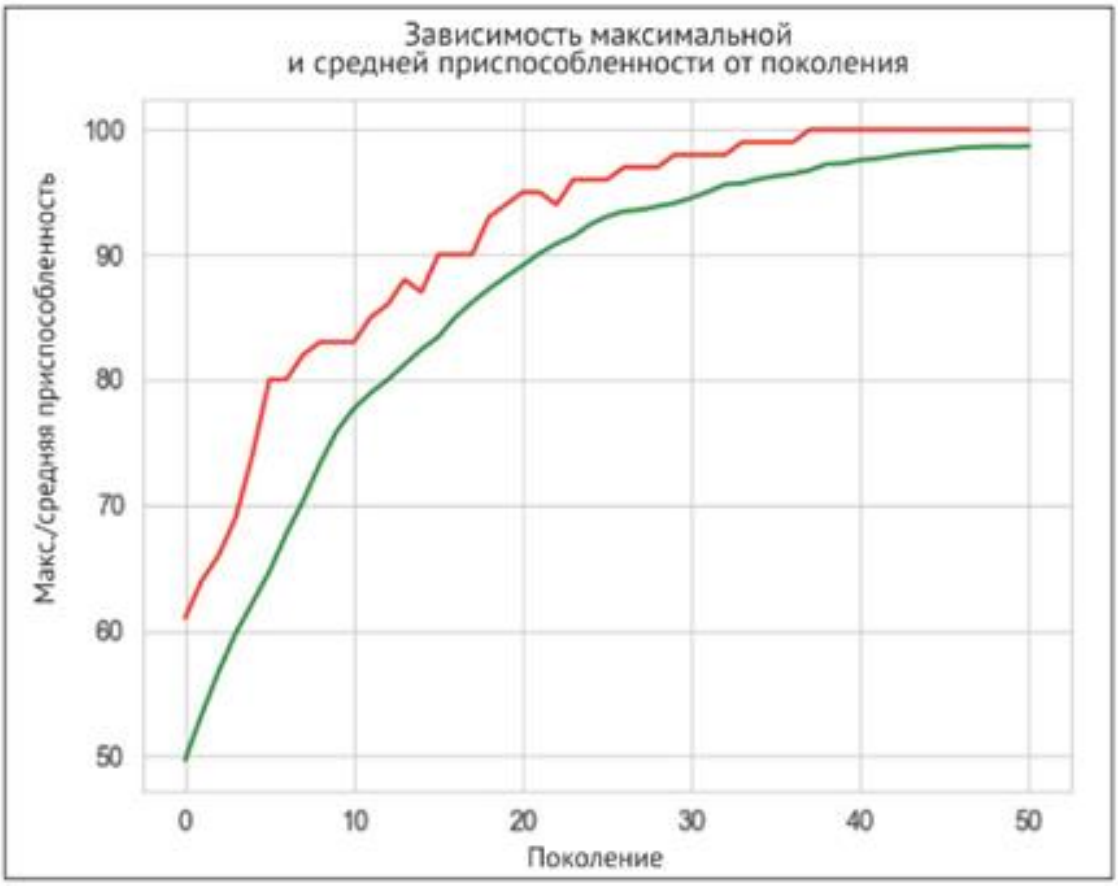
Статистика программы для решения задачи OneMax
после перехода на двухточечное скрещивание

Такое поведение типично для генетических алгоритмов с представлением хромосом двоичными строками, поскольку двухточечное скрещивание – более гибкий способ смешивания генов родителей по сравнению с одноточечным.

Оператор мутации

Снова откатим изменения, поскольку собираемся поэкспериментировать с оператором мутации, отвечающим за случайную модификацию потомка.

1. Для начала увеличим константу P_MUTATION до 0.9. Тогда график станет таким:



Статистика программы для решения задачи OneMax
после увеличения вероятности мутации до 0.9

На первый взгляд, результат удивительный, потому что при увеличении частоты мутаций алгоритм должен вести себя спорадически, но здесь этот эффект незаметен. Вспомним, однако, что у нашего алгоритма есть еще один параметр, относящийся к мутации, indpb – аргумент конкретного использованного в данном случае оператора мутации mutFlipBit:

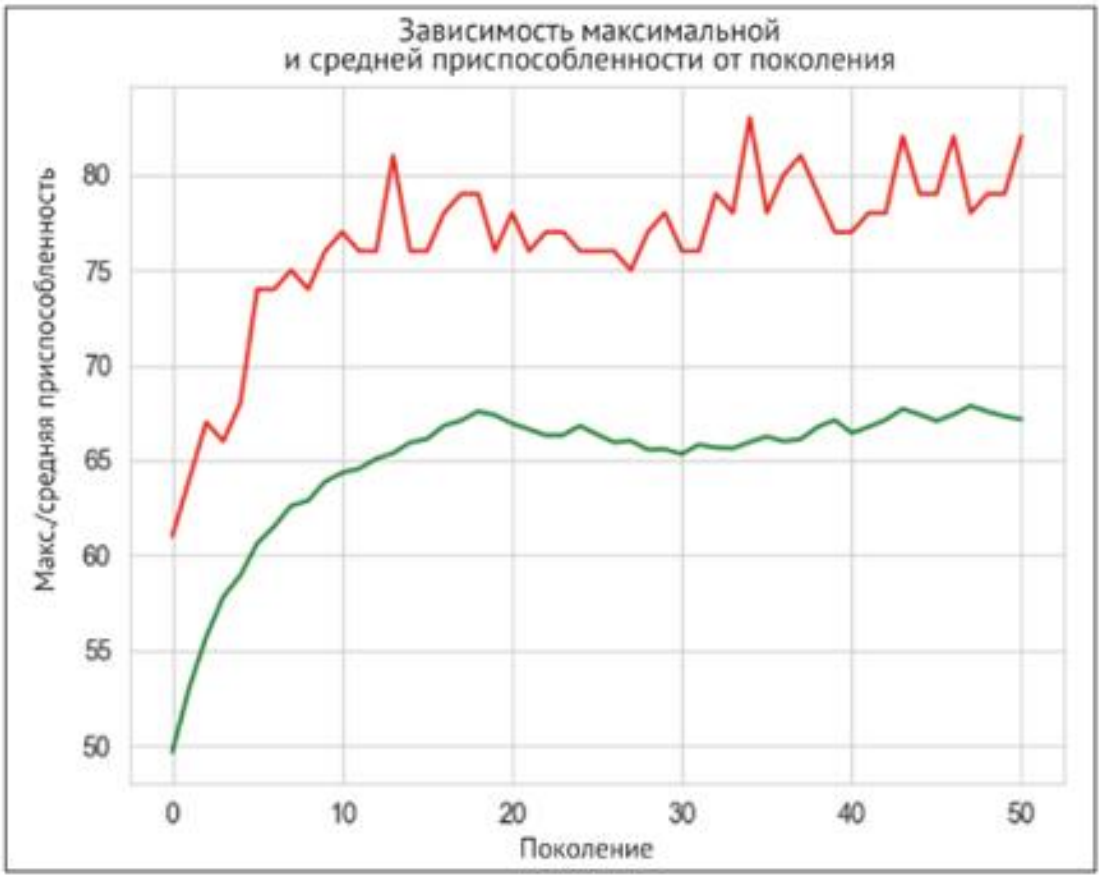
```
toolbox.register("mutate", tools.mutFlipBit, indpb=1.0/ONE_MAX_LENGTH)
```

Если P_MUTATION определяет вероятность мутации индивидуума, то indpb задает вероятность инвертирования каждого бита данного индивидуума. В нашей программе значение 1.0/ONE_MAX_LENGTH, т. е. в среднем в хромосоме мутирует один ген. По-видимому, в задаче OneMax с длиной индивидуума 100 это ограничивает эффект мутации независимо от значения P_MUTATION.

2. Увеличим значение `indpb` в десять раз:

```
toolbox.register("mutate", tools.mutFlipBit, indpb=10.0/ONE_MAX_LENGTH)
```

В результате поведение алгоритма становится спорадическим, как видно из следующего графика.



Статистика программы для решения задачи OneMax
после десятикратного увеличения вероятности мутации одного бита

Мы видим, что в начале работы алгоритму удается улучшать результаты, но довольно быстро он начинает осциллировать без видимого прогресса.

3. Если еще увеличить `indpb`, до `50.0/ONE_MAX_LENGTH`, то алгоритм становится совсем неустойчивым:



Статистика программы для решения задачи OneMax
после пятидесятикратного увеличения вероятности мутации одного бита

Из графика видно, что генетический алгоритм превратился, по сути дела, в случайный поиск – быть может, ему повезет наткнуться на лучшее решение, но никакого уверенного продвижения в сторону хороших решений не наблюдается.

Оператор отбора

Далее займемся оператором отбора. Сначала изменим размер турнира, чтобы посмотреть на совместный эффект этого параметра и вероятности мутации. А затем посмотрим, что будет, если заменить турнирный отбор правилом рулетки.

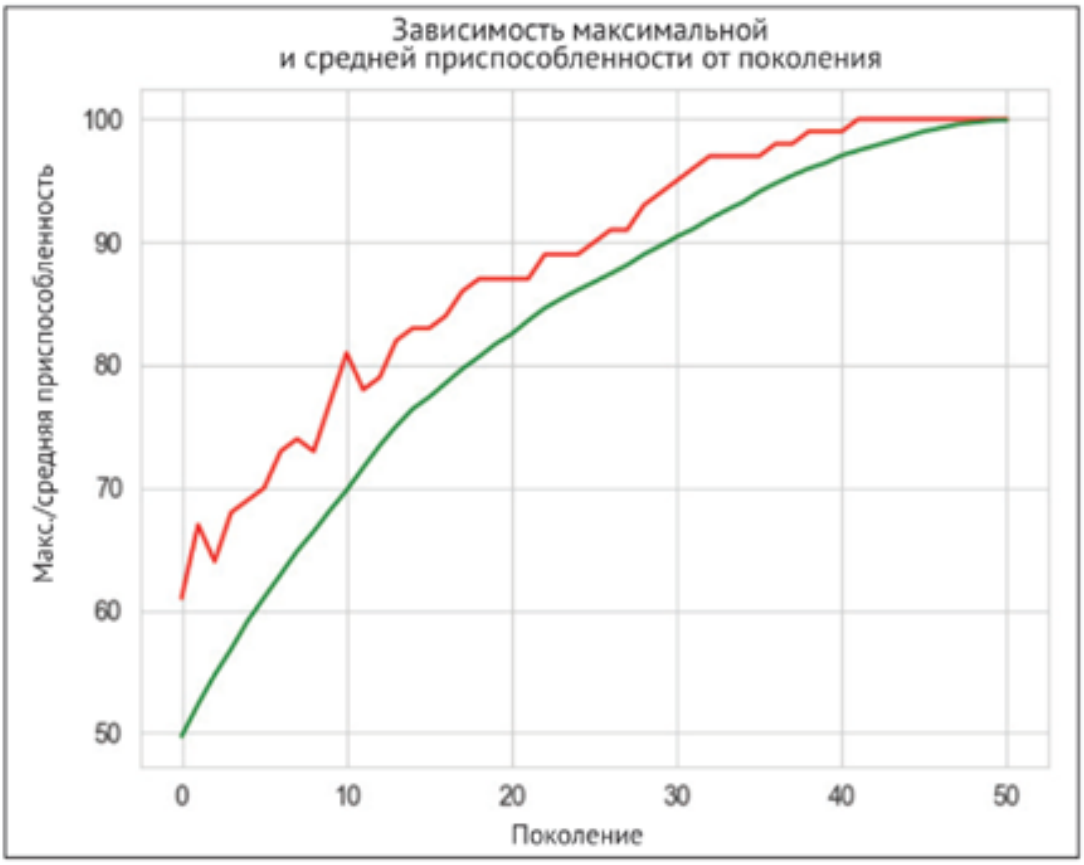
Размер турнира и его связь с вероятностью мутации

Снова восстановим первоначальные параметры программы, а затем внесем новые изменения.

1. Сначала сделаем параметр `tournamentSize` равным 2 (изначально он был равен 3):

   ```
   toolbox.register("select", tools.selTournament, tournsize=2)
   ```

   Не похоже, чтобы это как-то повлияло на поведение алгоритма.

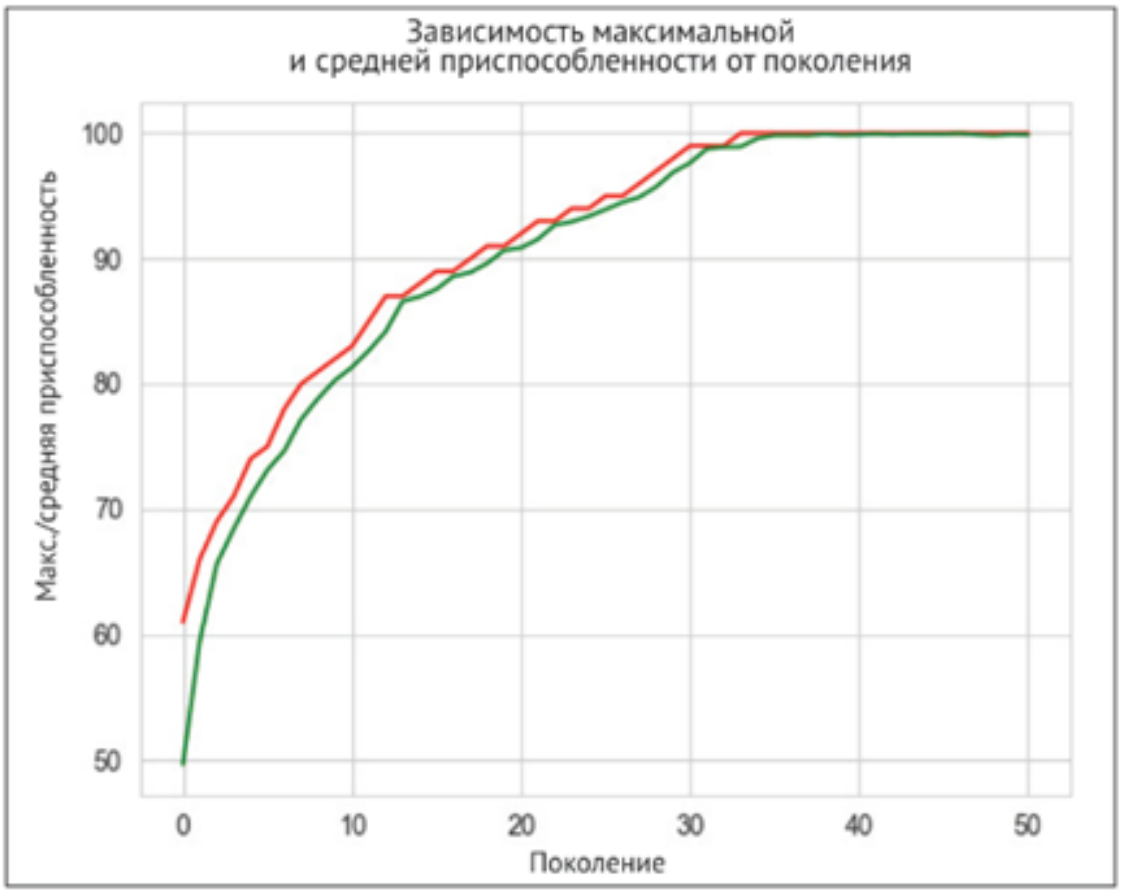


Статистика программы для решения задачи OneMax
после уменьшения размера турнира до 2

2. А что, если резко увеличить размер турнира, скажем, до 100? Поглядим:

   ```
   toolbox.register("select", tools.selTournament, tournsize=100)
   ```

   Алгоритм по-прежнему ведет себя хорошо и находит лучшее решение меньше, чем за 40 поколений. Правда, теперь графики максимальной и средней приспособленности очень похожи:



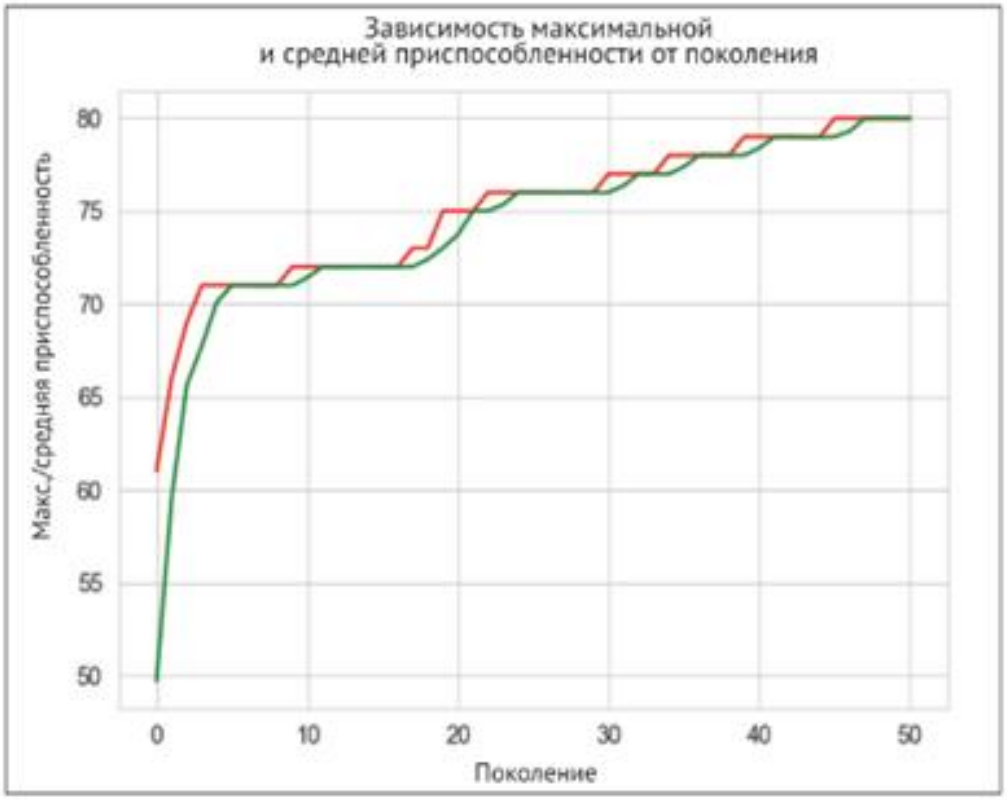
Статистика программы для решения задачи OneMax
после увеличения размера турнира до 100

Такое поведение объясняется тем, что при увеличении размера турнира шансы слабых индивидуумов отобраться падают, а лучшие решения занимают всю популяцию. В реальной задаче такое доминирование может привести к тому, что вся популяция будет насыщена неоптимальными решениями, а лучшее так и не будет найдено (этот феномен называется преждевременной сходимостью). Но в простой задаче OneMax подобной проблемы, похоже, не возникает. Возможно, это связано с тем, что оператор мутации привносит достаточно разнообразия, чтобы поиск решений продвигался в правильном направлении.

3. Чтобы проверить это объяснение, уменьшим вероятность мутации в 10 раз, до 0.01:

```
P_MUTATION = 0.01
```

Ели теперь прогнать алгоритм, то окажется, что результаты перестают улучшаться вскоре после запуска, а затем улучшаются гораздо медленнее и неравномерно. Итоговый результат гораздо хуже, чем после предыдущего прогона.



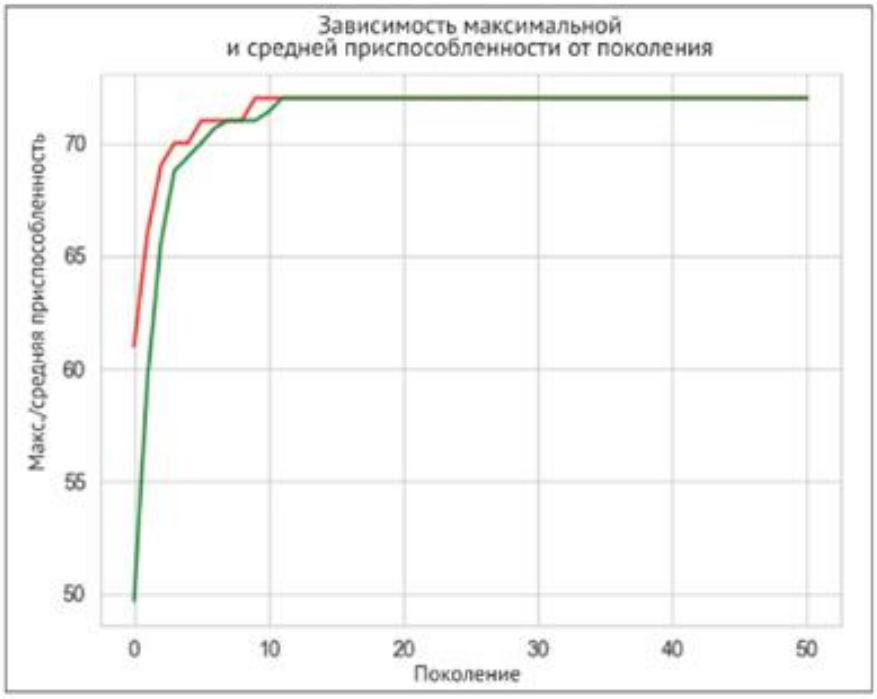
Статистика программы для решения задачи OneMax
с размером турнира 100 и вероятностью мутации 0.01

Это можно интерпретировать следующим образом: из-за большого размера турнира лучшие индивидуумы из начальной популяции занимают всю популяцию после небольшого числа смены поколений, и в течение этого времени мы наблюдаем быстрый рост обеих кривых. Но потом лишь случайная мутация в нужном направлении – инвертирование нулевого бита в единичный – приводит к появлению лучшего индивидуума, этому соответствуют редкие ступеньки на красном графике. Вскоре после этого найденный индивидуум снова занимает всю популяцию, и тогда зеленый график догоняет красный.

4. Чтобы довести эту ситуацию до крайности, уменьшим частоту мутаций еще в 10 раз:

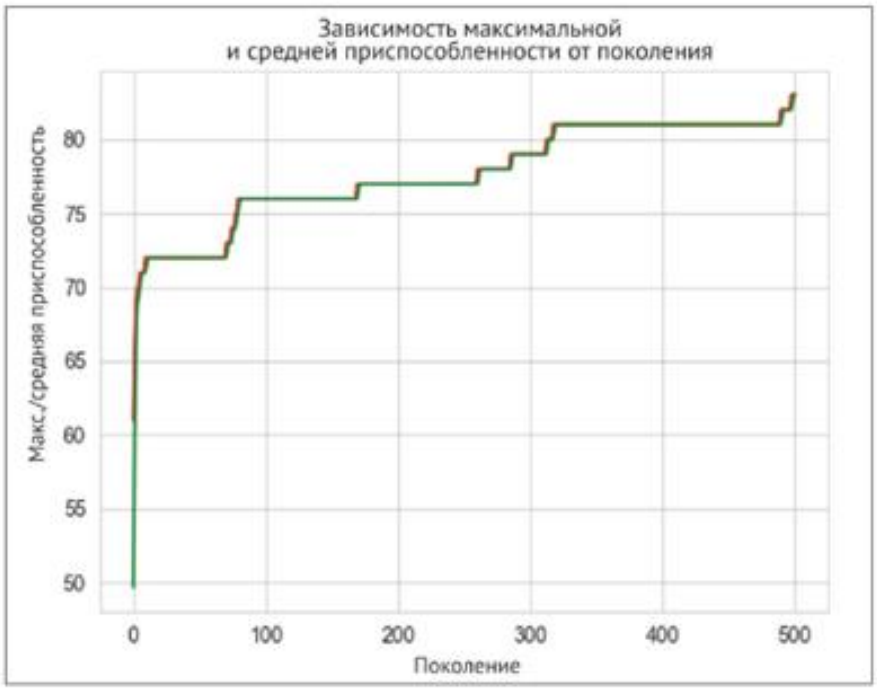
```
P_MUTATION = 0.001
```

Мы наблюдаем в целом такое же поведение, но, поскольку мутации случаются очень редко, улучшений немного и расстояние между ними больше.



Статистика программы для решения задачи OneMax
с размером турнира 100 и вероятностью мутации 0.001

5. Если увеличить количество поколений до 500, то это поведение станет еще нагляднее.

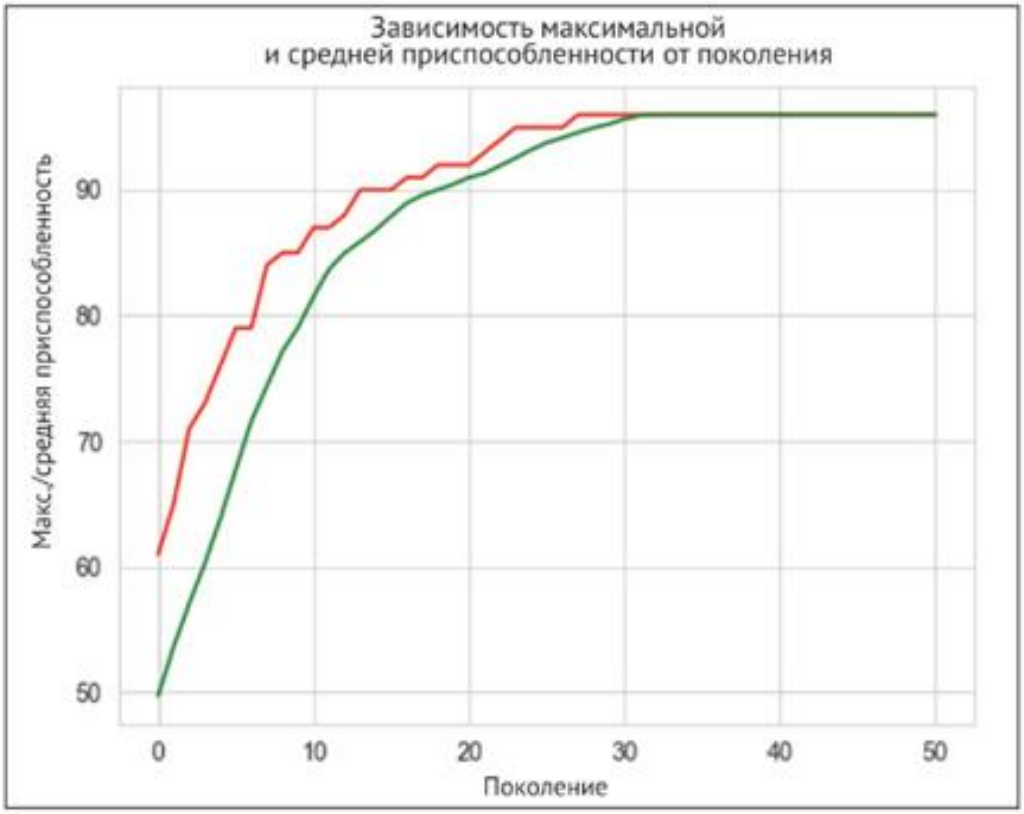


Статистика программы для решения задачи OneMax
с размером турнира 100 и вероятностью мутации 0.001
на протяжении 500 поколений

6. Просто из любопытства вернем размер турнира 3 и сделаем количество поколений равным 50, а низкую частоту мутаций оставим:

```
MAX_GENERATIONS = 50
toolbox.register("select", tools.selTournament, tournsize=3)
```

Получившийся график гораздо ближе к исходному:



Статистика программы для решения задачи OneMax
с размером турнира 3 и вероятностью мутации 0.001 на протяжении 50 поколений

Похоже, что теперь захват популяции тоже имеет место, но происходит гораздо позже, в районе поколения 30, когда наилучшая приспособленность близка к максимальному значению 100. Но более разумная частота мутаций позволила бы найти наилучшее решение, как то было при первоначальных параметрах.

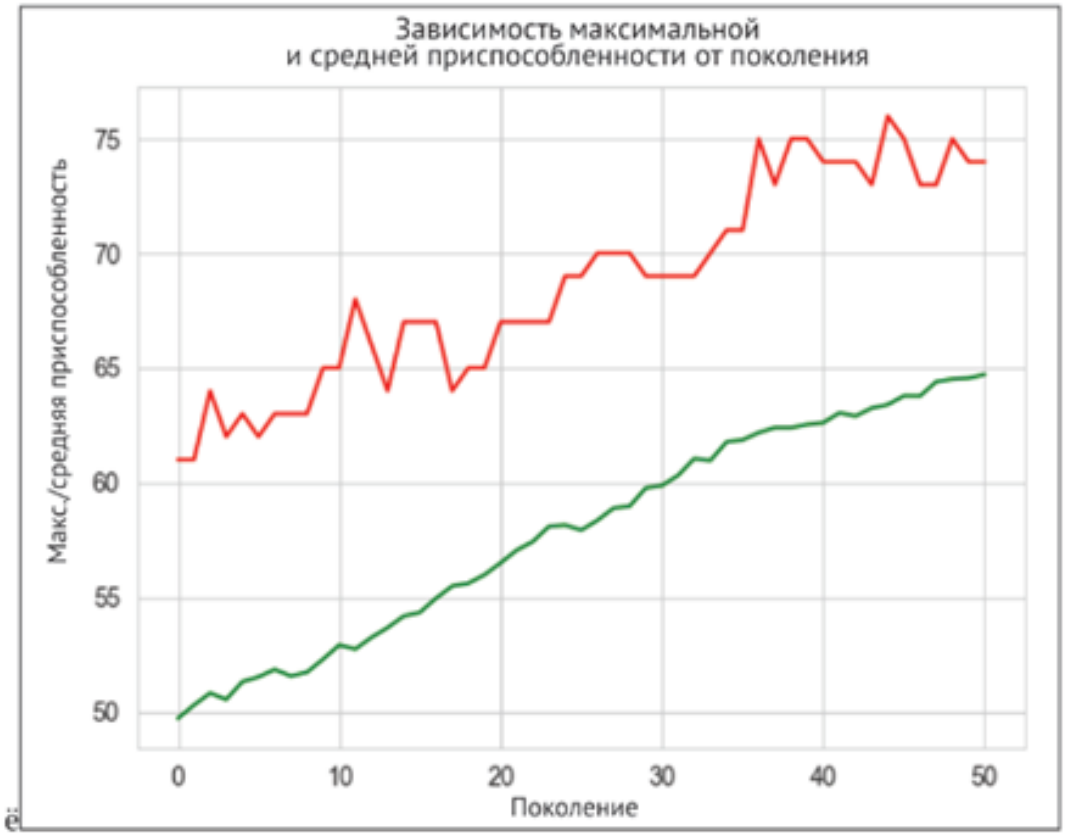
Отбор по правилу рулетки

Снова вернемся к первоначальным параметрам в предвкушении последнего эксперимента. Теперь мы заменим турнирный отбор отбором по правилу рулетки, описанным в главе 2. Делается это так:

```
toolbox.register("select", tools.selRoulette)
```

Это изменение плохо повлияло на результаты алгоритма. На графике видно, что есть много точек, в которых наилучшее решение оказывается забыто

и максимальная приспособленность уменьшается, по крайней мере временно, хотя средняя приспособленность монотонно возрастает. Это объясняется тем, что правило рулетки отбирает индивидуумов с вероятностью, пропорциональной их приспособленности; если различия между индивидуумами невелики, то у более слабых появляется больше шансов отобраться по сравнению с турнирным отбором.



Статистика программы при использовании правила рулетки

Для компенсации этого поведения можно применить элитистский подход. Тогда заданное количество лучших индивидуумов перейдет из текущего поколения в следующее без изменения, поэтому они не будут потеряны.